

# Deliverable #5: Service- Oriented Software Engineering

## **Part 1: Service Engineering Application**

### **1. Service Candidate Identification**

Based on the principles of Service-Oriented Architecture, the following services were identified for the system:

- Item Management Service
  - Classification: Business Service.
  - Justification: This service manages the core entity of the application (lost items). It is highly reusable and independent, as it can be utilized by students searching for lost property, or by security officers managing inventory.
- Claim Management Service
  - Classification: Business Service.
  - Justification: This service handles the lifecycle of item claims. It operates independently to track which student is claiming which item, allowing for clear separation of concerns.
- Claim Resolution Workflow
  - Classification: Coordination Service (Composite).
  - Justification: This service orchestrates the Item and Claim business services to execute a complete real-world process without requiring the client to make multiple manual API calls.

## 2. Service Requirements Specification

### For the Item and Claim Business Services:

- **Functional Requirements:** The system must allow authorized users to Create, Read, Update, and Delete (CRUD) records in the SQL Server database.
- **Non-Functional Requirements:** The API must be stateless, respond within 500ms, and ensure data integrity between parent and child tables.
- **Inputs/Outputs & Constraints:** The services must accept and return data exclusively in application/json format. The system must enforce constraints, such as ensuring an UploadedByUserID is present when creating an item.

## 3. Service Interface Design

The APIs were designed following RESTful principles :

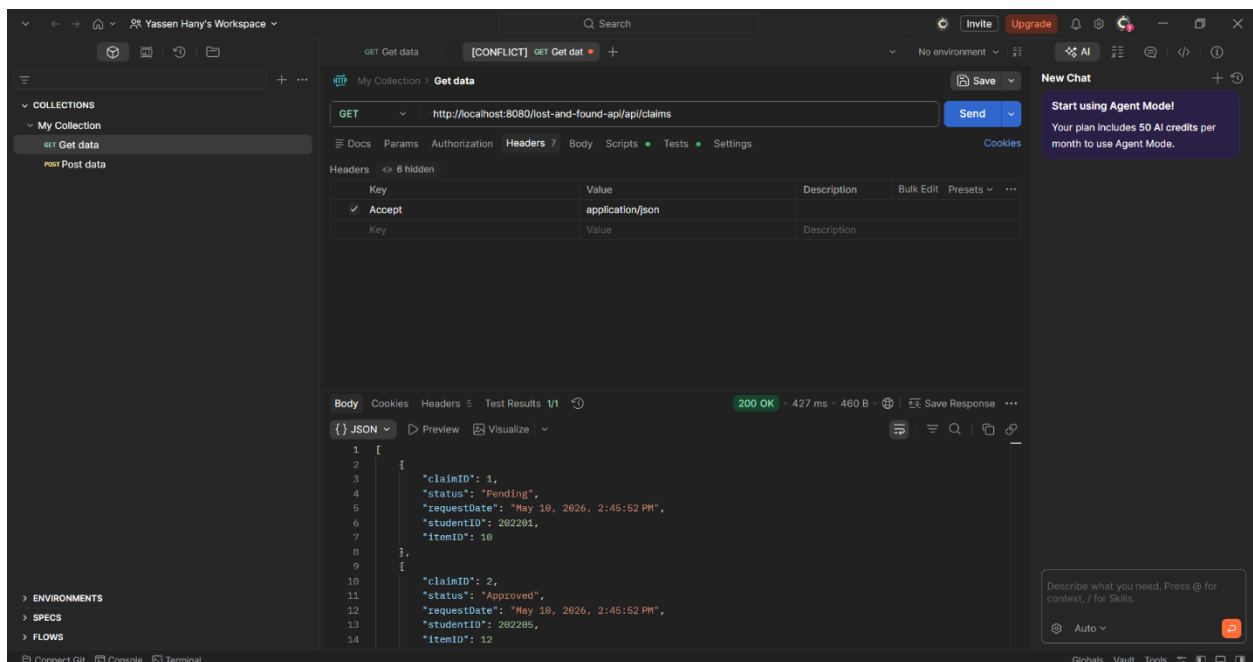
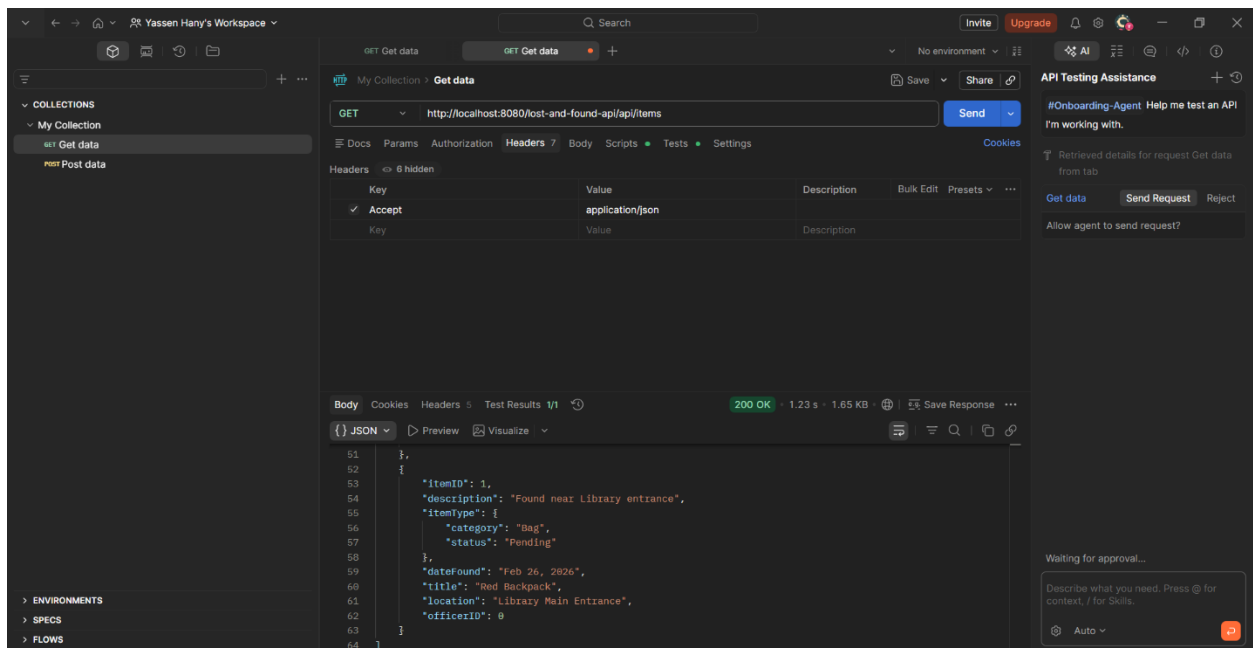
- **Resources & Endpoints:** /api/items and /api/claims.
- **HTTP Methods:** GET (Retrieve), POST (Create), PUT (Update), and DELETE (Remove).
- **Format:** JSON Request and Response payloads.

## Part 2: REST API Implementation & Testing

The following screenshots demonstrate successful Postman tests for the implemented REST APIs.

### 1. GET APIs (Retrieve Data)

**Endpoint:** GET /api/items & GET /api/claims



## 2. POST APIs (Create Data)

Endpoint: POST /api/items & POST /api/claims

The screenshot shows the Postman interface with a workspace named "Yassen Hany's Workspace". The left sidebar shows a collection named "My Collection" with a sub-collection "Get data". The main panel displays a POST request to the endpoint `http://localhost:8080/lost-and-found-api/api/items`. The request body is raw JSON, containing an item object with fields: `title`, `description`, `category`, `location`, `status`, `officerName`, and `officerID`. The response status is `201 Created`, and the response body is a JSON object with fields: `itemID`, `description`, `dateFound`, `title`, `location`, `officerName`, and `officerID`.

```
1 {
2   "title": "Black Laptop Charger",
3   "description": "Found plugged into the wall in room 3B2.",
4   "category": "Electronics",
5   "location": "Room 3B2",
6   "status": "Found",
7   "officerName": "Yassin",
8   "officerID": 101
9 }
```

```
1 {
2   "itemID": 0,
3   "description": "Found plugged into the wall in room 3B2.",
4   "dateFound": "May 10, 2026, 2:49:44 PM",
5   "title": "Black Laptop Charger",
6   "location": "Room 3B2",
7   "officerName": "Yassin",
8   "officerID": 101
9 }
```

The screenshot shows the Postman interface with the same workspace. The main panel displays a POST request to the endpoint `http://localhost:8080/lost-and-found-api/api/claims`. The request body is raw JSON, containing a claim object with fields: `studentID`, `itemID`, and `status`. The response status is `201 Created`, and the response body is a JSON object with fields: `claimID`, `status`, `requestDate`, `studentID`, and `itemID`.

```
1 {
2   "studentID": 202401,
3   "itemID": 15,
4   "status": "Pending"
5 }
```

```
1 {
2   "claimID": 3,
3   "status": "Pending",
4   "requestDate": "May 10, 2026, 2:51:00 PM",
5   "studentID": 202401,
6   "itemID": 15
7 }
```

### 3. PUT APIs (Update Data)

Endpoint: PUT /api/items/{id} & PUT /api/claims/{id}

The screenshot shows the Postman interface for a PUT request to the endpoint `http://localhost:8080/host-and-found-api/api/items/1`. The request body is a JSON object:

```
{
  "title": "Black Laptop Charger",
  "description": "Found plugged into the wall in room 382.",
  "category": "Electronics",
  "location": "Security Desk",
  "status": "Found",
  "officerName": "Yassin",
  "officerID": 101
}
```

The response is a 200 OK status with a 45 ms response time and 362 B of data. The response body is a JSON object:

```
{
  "itemID": 1,
  "description": "Found plugged into the wall in room 382.",
  "title": "Black Laptop Charger",
  "location": "Security Desk",
  "officerName": "Yassin",
  "officerID": 101
}
```

The screenshot shows the Postman interface for a PUT request to the endpoint `http://localhost:8080/host-and-found-api/api/claims/1`. The request body is a JSON object:

```
{
  "studentID": 202401,
  "itemID": 15,
  "status": "Approved"
}
```

The response is a 200 OK status with a 5 ms response time and 251 B of data. The response body is a JSON object:

```
{
  "claimID": 1,
  "status": "Approved",
  "studentID": 202401,
  "itemID": 15
}
```

## 4. DELETE APIs (Remove Data)

**Endpoint:** DELETE /api/items/{id} & DELETE /api/claims/{id}

The screenshot shows the Postman interface for a DELETE request. The URL is `http://localhost:8080/lost-and-found-api/api/items/1`. The method is set to **DELETE**. The headers tab is active, showing `Accept: application/json` and `Content-Type: application/json`. The response is **204 No Content** with a status of **0/1**. The response body is empty. The left sidebar shows the collection structure with `GET Get data` and `POST Post data`. The right sidebar shows a **New Chat** button and a message about the Agent Model.

| Key            | Value            | Description | Bulk Edit | Presets |
|----------------|------------------|-------------|-----------|---------|
| ✓ Accept       | application/json |             |           |         |
| ✓ Content-Type | application/json |             |           |         |

Body: Cookies Headers 3 Test Results 0/1 204 No Content 81 ms · 112 B · Save Response

Raw Preview Visualize

1

The screenshot shows the Postman interface for a DELETE request. The URL is `http://localhost:8080/lost-and-found-api/api/claims/1`. The method is set to **DELETE**. The headers tab is active, showing `Accept: application/json` and `Content-Type: application/json`. The response is **204 No Content** with a status of **0/1**. The response body is empty. The left sidebar shows the collection structure with `GET Get data` and `POST Post data`. The right sidebar shows a **New Chat** button and a message about the Agent Model.

| Key            | Value            | Description | Bulk Edit | Presets |
|----------------|------------------|-------------|-----------|---------|
| ✓ Accept       | application/json |             |           |         |
| ✓ Content-Type | application/json |             |           |         |

Body: Cookies Headers 3 Test Results 0/1 204 No Content 3 ms · 112 B · Save Response

Raw Preview Visualize

1

## Part 3: Service Composition

### Description of Composite Service

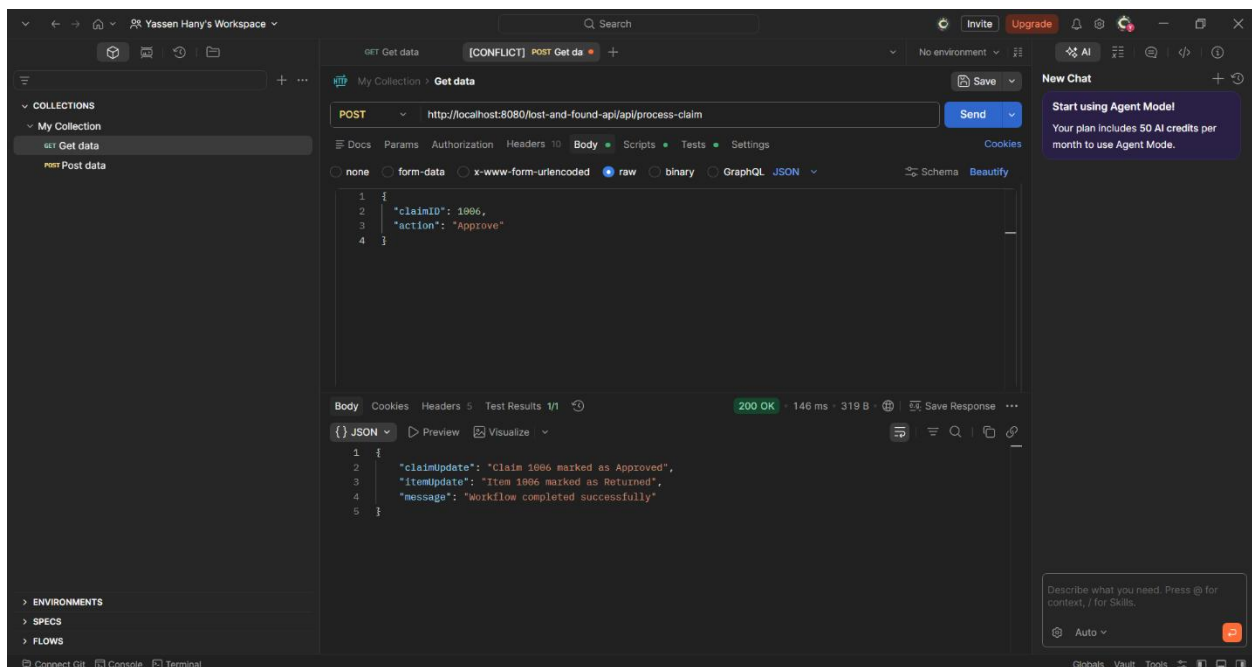
To represent a complete business workflow, a Composite Service was implemented at the endpoint `/api/process-claim`.

### Workflow Explanation

When a security officer reviews a claim, updating the claim status alone is insufficient; the system must also secure the physical item. This composite service orchestrates two separate APIs in a single transaction:

1. **API Action 1:** Updates the Claim database record to "Approved" or "Rejected".
2. **API Action 2:** Locates the corresponding Item record and automatically updates its status to "Returned" or "Found".

### Composite Service Test





## Part 4: API Documentation

The following is the OpenAPI Specification documenting the RESTful services

